DBMS Course Project
Full-Stack Database Application

1. Project Objective

The objective of this project is to design and develop a complete full-stack database application for a realistic problem domain. Each team will identify an application involving non-trivial data-management requirements and build a functional system consisting of a well-designed relational database, an application/backend layer that interacts with it, and a usable front-end interface through which users can perform meaningful operations.

While the project requires you to build an application, Database design and effective use of DBMS concepts remain the core of the project. A visually sophisticated application built over a poorly designed database will not constitute a strong DBMS project, and will be awarded marks accordingly.

2. Choice of Application

Teams may select any sufficiently rich application domain. Examples include:

- ticket or event booking,
- food delivery or restaurant management,
- course registration / learning-management systems,
- healthcare and appointment management,
- e-commerce marketplaces,
- inventory and supply-chain management,
- travel and reservation systems,
- professional or social networking,
- music/video streaming,
- sports or tournament management,
- campus/hostel/facility management,
- service marketplaces,
- or any other suitable application the team proposes.

You may study existing applications like BookMyShow, RedBus, Moodle, Swiggy, Urban Company, Zomato, Coursera, Practo, BigBasket, IRCTC, Amazon, Flipkart, Netflix, Spotify, LinkedIn, Instagram for inspiration.

You are not expected to reproduce an entire existing application. Instead, identify a sufficiently interesting subset of functionality that requires meaningful database design and operations.

3. Project Requirements

A. Requirements Analysis and ER/EER Design

Clearly identify the problem being addressed; intended users of the system; important application functionality; assumptions; and important business rules and constraints.

Develop an appropriate ER/EER model identifying entities and attributes, primary/candidate keys, relationships, relationship cardinalities, participation constraints, weak entities (where applicable), and any other relevant modelling constructs. The ER/EER model should then be mapped to an appropriate relational schema.

B. Relational Schema and Integrity Constraints

The relational schema should demonstrate sound database-design principles. You should identify appropriate primary and foreign keys; select suitable domains and data types; identify and enforce integrity constraints; minimize unnecessary redundancy; examine the schema with respect to appropriate normal forms; and justify important schema-design decisions.

Constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK should be enforced by the DBMS wherever appropriate.

C. Database Implementation and SQL

The database may be implemented using any suitable RDBMS including PostgreSQL, MySQL, MariaDB, SQL Server, Oracle, SQLite, or another appropriate relational DBMS.

Your project should demonstrate meaningful use of SQL and DBMS functionality. Depending on your application, this may include: multi-table joins; aggregate queries and grouping; nested or correlated queries; Common Table Expressions (CTEs); views; transactions; triggers; stored procedures/functions; indexes; database-level integrity constraints; and other relevant DBMS concepts covered in the course.

D. Full-Stack Application

You are free to select an appropriate technology stack. The overall system should follow: Front End ↔ Application/Backend ↔ Relational Database. The front end may be web-based, mobile, desktop, or another suitable interface.

The application should support insertion, modification, and deletion of data (where appropriate); searching, filtering, and retrieval; operations involving multiple relations; and meaningful reports, summaries, dashboards, or application-specific queries.

4. Expected Project Scope

The following indicate expected scope. Creating additional tables, inserting thousands of rows, or adding unnecessary triggers, views, indexes, or procedures will not automatically result in higher marks.

- Approximately 8–12 meaningful relations (tables). A substantially smaller schema should be justified based on your application's requirements.
- Sufficient data to demonstrate the application, normally at least 500 records in the database.
- At least 3 end-to-end application workflows.
- At least 5 non-trivial SQL queries using joins, aggregation, grouping, subqueries, and/or CTEs.
- Appropriate definition of primary keys, foreign keys, uniqueness, nullability, and other application-specific constraints.
- Use of at least 3 additional DBMS features like transactions, views, triggers, stored procedures/functions, indexes.

Your three workflows should preferably exercise different aspects of the database: one may involve a transaction or coordinated change across multiple relations; another a complex retrieval, search, aggregation, or reporting operation; and another a demonstration of integrity or constraint enforcement.

You may use AI tools in the course of building this project; however, such use must be judicious and not a substitute for genuine understanding. Every team member is expected to fully understand the ER/EER diagram and overall database design, and should be prepared to explain and justify it during the individual viva.

The use of any unethical means, including plagiarism, misrepresentation of work, or submission of work not genuinely understood or produced by the team, will be treated as a violation of academic integrity and may result in deduction of marks, resubmission, or a zero for the component.

5. In-Lab Demonstration and Evaluation

There will be 2 evaluation points - one at mid-sem and the other final project submission.

The mid-sem evaluation will be through a project submission that includes the following details

Component
Database Design & Schema (ER/EER, relational mapping, keys, constraints, normalization)
High Level Application Design and Technology Stack, Progress and team member contributions

The final project submission will be evaluated through an in-lab live demonstration and viva. Demonstrate the project as an integrated, database-backed application not as a collection of independently implemented features. The structure of the demonstration maps directly to the evaluation criteria below.

Component
Application Workflows & Database Integration (3+ end-to-end workflows)
SQL & DBMS Concepts (5+ non-trivial queries, 3+ DBMS features)
Application Demo and Questions (Viva)

6. Project Submission

There will be two project submissions: one at the mid-sem evaluation and one prior to the final project demo. Single submission should be made per team.

Mid-Sem Submission - Each team must submit:

- Database design document (PDF max 5 pages): ER/EER diagram, relational schema, primary and foreign keys, important integrity constraints, assumptions, and a brief normalization/design justification.
- Application design: proposed application workflows, technology stack, and high-level application architecture.
- GitHub repository: work completed up to the mid-sem evaluation, with a brief README describing the project, current progress, and each member's contribution.

Final Submission - Prior to the final project demo, each team must submit a link to a GitHub repository containing the complete project. The repository should contain:

- Complete application: front-end source code, backend/application source code, and any configuration files required to run it.
- Database scripts: schema/table creation, primary and foreign keys, integrity constraints, views, triggers, stored procedures/functions, indexes, and data population/sample-data scripts.
- README.md: project title, brief description, names and IDs of all team members, technology stack used, setup/run instructions, a brief description of the major application workflows, and each member's contribution.
- Database design document (PDF max 5 pages): final ER/EER diagram, relational schema, important integrity constraints, assumptions, a brief normalization/design justification, and important schema-design decisions.
- Reproducibility: sufficient code, scripts, sample data, and instructions for the demonstrated application to be reproduced. The repository must correspond to the final version demonstrated in class.

The GitHub repository and its documentation support the evaluations — they are not separate evaluation components. Evaluators may inspect the repository, scripts, or code and ask you to locate, explain, modify, or execute relevant parts during the demonstration and viva.

7. Timelines

18-Sep (Fri)	Project Statements Released
18-Sep (Fri)	Team Assignments Released
11-Oct (Sun)	Mid-Sem Submission (Details as per Section 6 above)
15-Nov (Sun)	Final Project Submissions (Details as per Section 6 above)
17-19 Nov (Tue/Thu)	Final Project Demo Presentations during lab sessions (Details as per Section 5 above)

8. General Evaluation Principle

The objective of this project is not simply to build an attractive application or to maximize the number of database objects. A visually polished application will not receive high marks if the underlying database is poorly designed or DBMS concepts are used superficially. Likewise, creating a large number of tables, queries, triggers, views, indexes, or stored procedures will not automatically raise your score.

Projects are evaluated on how effectively you translate: Application Requirements → ER/EER Design → Relational Schema → Integrity Constraints & DBMS Concepts → Application Workflows → Working Full-Stack System.

The strongest projects demonstrate a well-designed relational database; meaningful use of DBMS concepts; correct integration between application and database; meaningful end-to-end workflows; a convincing live demonstration; and a sound understanding of the design and implementation by every member of the team.

9. Team Formation

Team allocation will be done according to lab sessions and ID numbers. Changes to teams will not be encouraged, the goal is for you to work together to deliver this project.